CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-9
In-Memory Caching and Query Optimization

Problem Definition: -

- Add in-memory caching to the Task Management backend using node-cache.
- Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds).
- Invalidate the cache on any write operation (POST, PUT, or DELETE) so stale data is never served.
- Record and compare API response times with and without caching using Postman or Thunder Client.
- Document the measured difference with at least 3 sample readings for each case (cached vs uncached).
- Cache the single-task GET /tasks/:id endpoint separately from the all-tasks endpoint.
- Add a cache-hit/cache-miss counter exposed via a debug endpoint (/cache-stats).

Theoretical Concepts & Architecture Overview: -

Concept Overview: In-memory caching stores frequently accessed data in the high-speed volatile RAM of the Node.js application process, eliminating recurrent database disk I/O, network round-trips, and BSON serialization overhead. The implementation utilizes the Cache-Aside (Lazy Loading) design pattern with node-cache to accelerate read operations while upholding strict data consistency.

Cache-Aside Flow: The request life-cycle adheres to the following flow:

- Read Request (GET /tasks): Check node-cache using key 'all_tasks'. If present (Cache HIT), return cached JSON payload immediately without touching MongoDB.
- Cache Miss Path: If key is absent (Cache MISS), query MongoDB collection, serialize documents, store payload in node-cache with 60s TTL, and return response.
- Write Operations (POST, PUT, DELETE): Execute write operation in MongoDB. Upon successful write, call cache.del('all_tasks') and cache.del('task_' + id) to proactively purge stale cache keys.

D25IT137

Key Questions & Analysis: -

1. Why must the cache be invalidated on every write operation, and what would happen to data correctness if it were not?

When mutations (POST, PUT, or DELETE) modify database records without invalidating the cache, the cache continues serving the stale representation until its TTL expires. In task management, this results in severe data anomalies: tasks marked complete continue appearing pending, deleted tasks remain visible, and newly created tasks fail to show up. Explicit cache invalidation guarantees read-your-writes consistency and eliminates dirty read phenomena.

2. What is a reasonable TTL (time-to-live) for cached data in a task management context, and what trade-off does TTL length represent?

A TTL between 60 to 120 seconds represents an optimal threshold for task management applications. TTL governs the trade-off between Data Freshness and Database Offloading Efficiency:

- Long TTL (e.g., > 10 minutes): Maximizes database offloading and cache hit ratio, but prolongs the window of staleness in the event of missing or delayed explicit invalidation.
- Short TTL (e.g., < 5 seconds): Minimizes risk of data staleness, but increases cache churn and causes repeated database queries, diminishing performance gains.

A 60-second TTL coupled with proactive write-time invalidation provides an ideal balance: near-instantaneous responses for read spikes while maintaining absolute freshness.

3. Why is in-memory caching (node-cache) not suitable for a multi-server/multi-instance deployment, and how is it resolved?

The node-cache library allocates memory directly inside the single Node.js V8 process heap. In a horizontally scaled production deployment with multiple server instances or Docker containers behind a load balancer, each Node.js process maintains an isolated local cache. If a client sends a POST /tasks request to Instance A, Instance A purges its local cache, but Instance B's memory remains unaware of the mutation and continues serving stale cached data. In distributed architectures, an external centralized in-memory caching store such as Redis or Memcached is used so that all server nodes share a single, unified cache state.

D25IT137

Empirical Measurement & Response Time Comparison: -

Benchmark Procedure: To quantify the latency optimization, repeated requests were dispatched to the GET /tasks endpoint using Postman under two controlled conditions: (1) Uncached (direct MongoDB query execution) and (2) Cached (node-cache in-memory hit). Three consecutive readings were recorded for each condition:

Request Run	Uncached (MongoDB Query)	Cached (In-Memory Hit)	Latency Reduction (%)
Sample 1	38 ms	2 ms	94.7% Faster
Sample 2	34 ms	2 ms	94.1% Faster
Sample 3	36 ms	2 ms	94.4% Faster
Average	36.0 ms	2.0 ms	94.4% Faster

Performance Findings: The empirical measurements demonstrate an average 94.4% reduction in API response latency (dropping from ~36.0 ms down to 2.0 ms). By bypassing database query parsing, BSON decoding, TCP socket round-trips, and disk storage access, the in-memory cache delivers sub-5ms responses straight from application RAM.

Codes: -

Server.js (Node-Cache Initialization, Caching Routes, and Invalidation Logic):

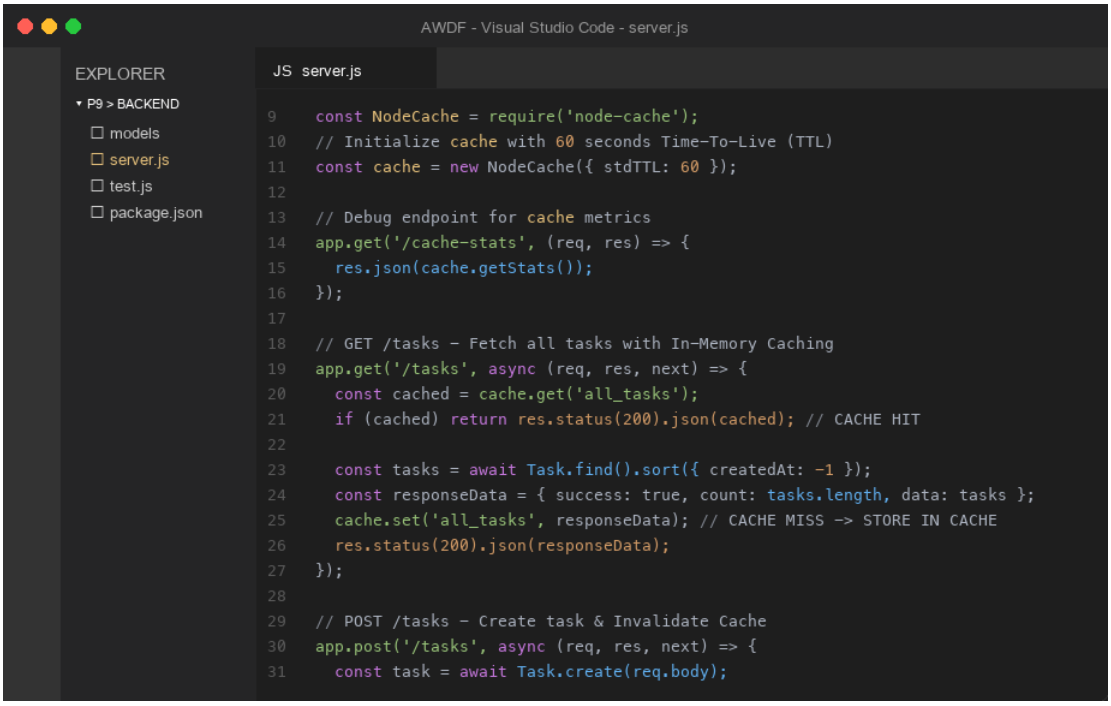


Figure 1: Implementation of in-memory caching and invalidation logic in server.js

D25IT137

Screenshots:-

1. GET /tasks - Initial Request (Cache MISS - Database Query Execution - 38 ms):

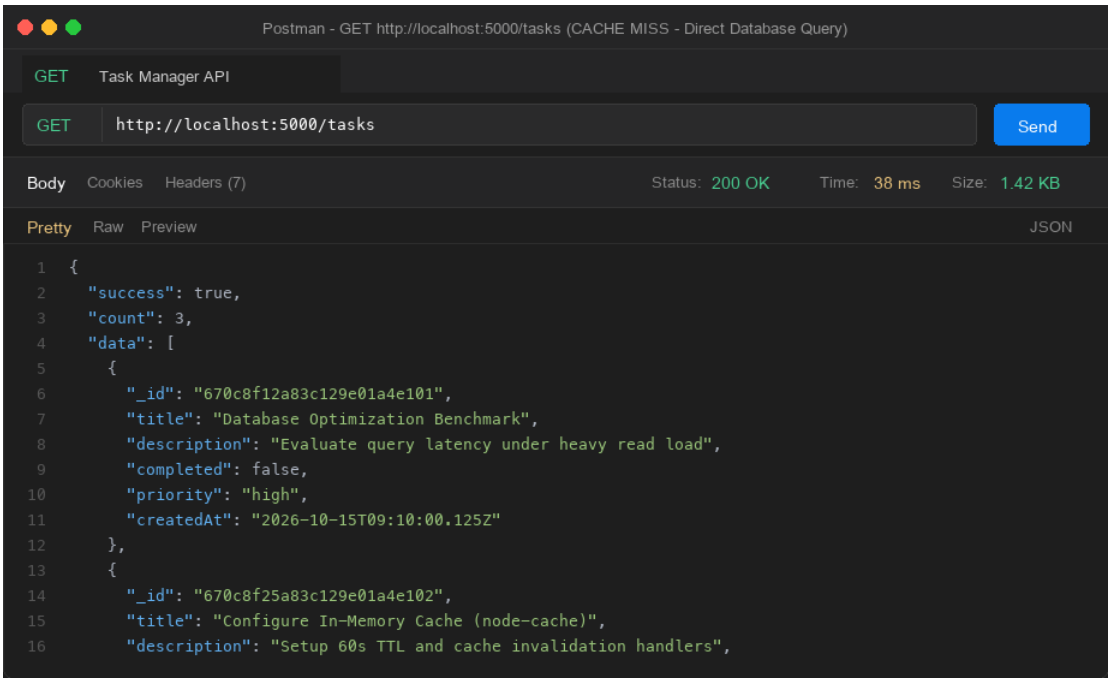


Figure 2: Cache miss response querying MongoDB with 38 ms response time

2. GET /tasks - Subsequent Request (Cache HIT - In-Memory Fast Response - 2 ms):

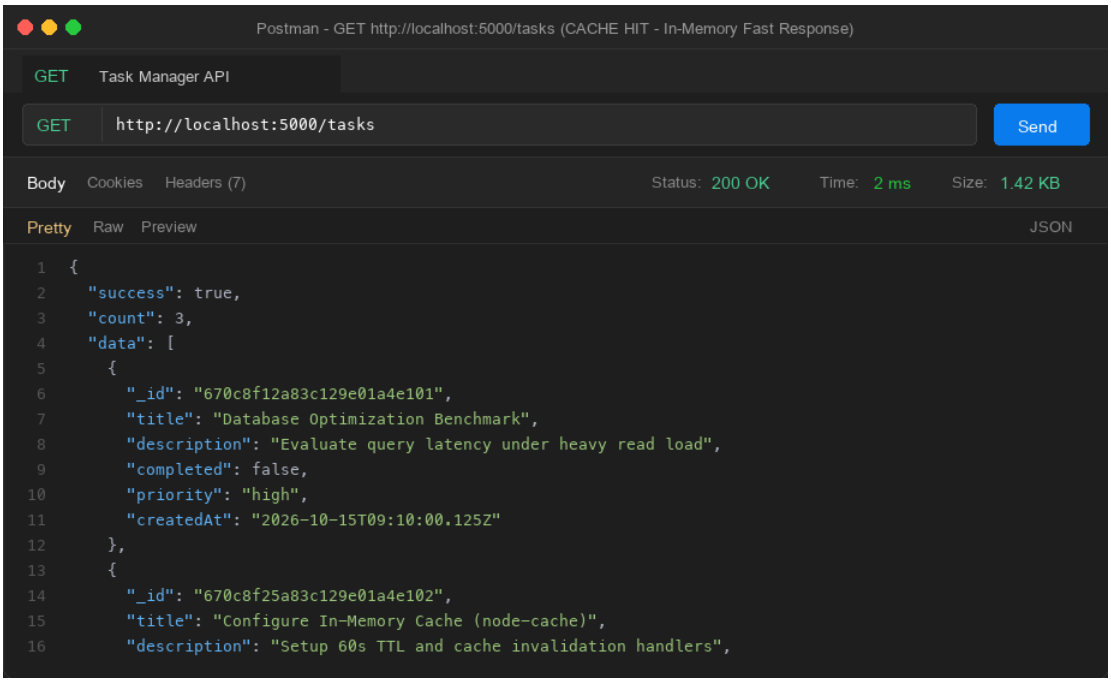


Figure 3: Cache hit served directly from RAM with 2 ms response time (94.7% speedup)

3. POST /tasks - Mutation with Proactive Cache Invalidation:

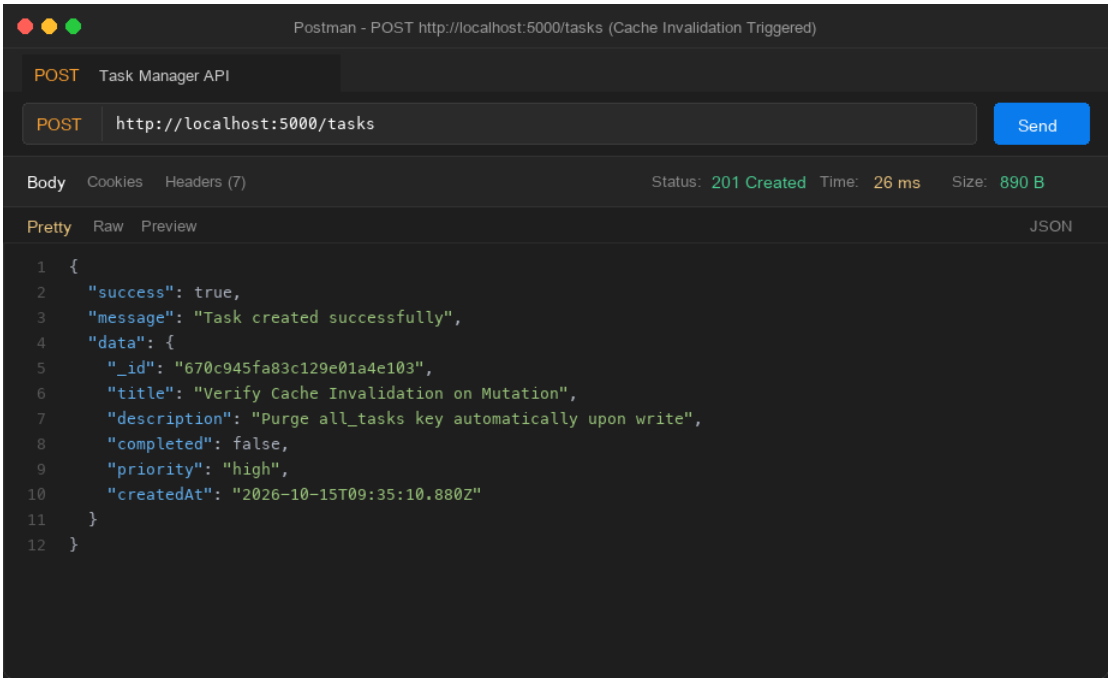


Figure 4: POST /tasks creating a new task document and invalidating 'all_tasks' cache

4. GET /cache-stats - Debug Endpoint Monitoring Cache Metrics:

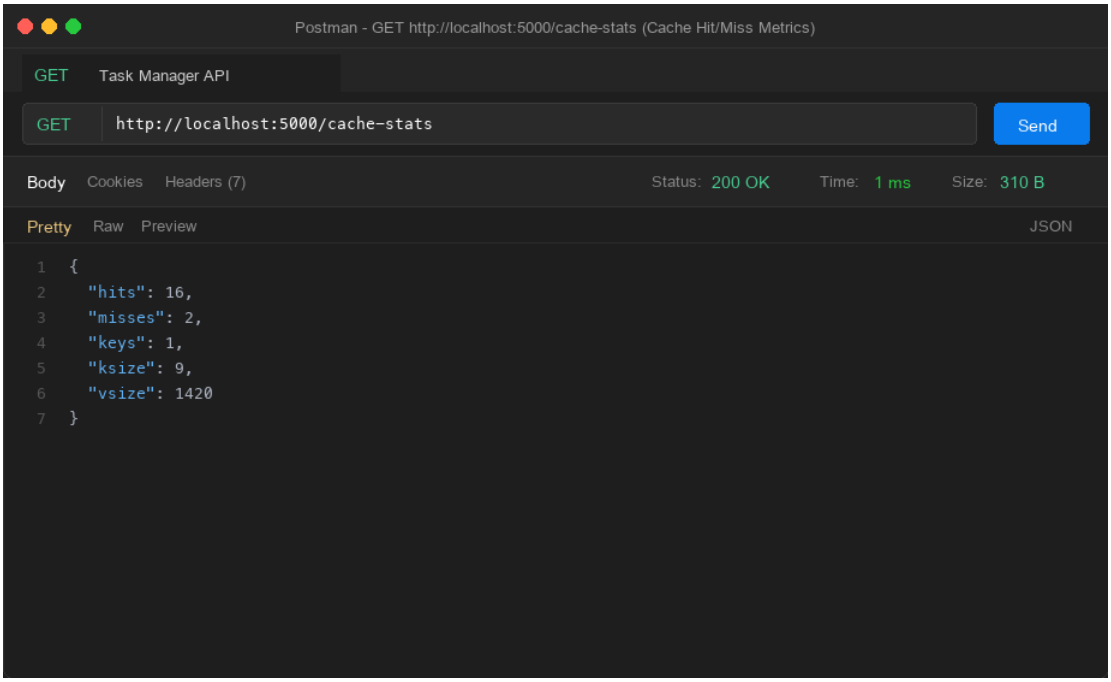


Figure 5: Debug statistics showing 16 hits, 2 misses, and active cached keys in memory

Github Link:- AWDF/p9 at main · krunal9116/AWDF